

UNIVERSIDADE FEDERAL DE SERGIPE
CENTRO DE CIÊNCIAS EXATAS E TECNOLOGIA
DEPARTAMENTO DE COMPUTAÇÃO

THYFANNY DA CRUZ SILVA
FLAVIO DA SILVA INACIO DOS SANTOS

PROJETO AZULON
RELATÓRIO TÉCNICO SOBRE O MINIMAX E SUAS ESTRATÉGIAS

SÃO CRISTÓVÃO - SE

2026

SUMÁRIO

1. INTRODUÇÃO.....	3
2. O JOGO: TRADIÇÃO VS. ADAPTAÇÃO.....	3
2.1. O JOGO DE DAMAS CLÁSSICO.....	3
2.2. VARIANTE "DAMAS DOS ELEMENTOS" (6X6).....	3
3. DINÂMICA DE PONTUAÇÃO E ANÁLISE DE CENÁRIOS.....	4
3.1. DISTRIBUIÇÃO DE PESOS E ALEATORIEDADE.....	4
3.2. COMENTÁRIOS SOBRE A DINÂMICA DE JOGO.....	5
4. DETALHAMENTO DA IMPLEMENTAÇÃO.....	6
4.1. ALGORITMO DE BUSCA COMPETITIVA (MINIMAX).....	6
4.2. FUNÇÃO DE AVALIAÇÃO HEURÍSTICA (HEURISTIC EVALUATION).....	9
4.3. CONDIÇÕES DE TERMINALIDADE (GAME OVER).....	10
4.4. DETERMINAÇÃO DO VENCEDOR POR PONTUAÇÃO PONDERADA.....	11
5. INTERFACE GRÁFICA E EXPERIÊNCIA DO USUÁRIO.....	12
5.1. AMBIENTE DE PARTIDA E MONITORAMENTO.....	12
5.2. INDICAÇÃO DE MOVIMENTOS POSSÍVEIS.....	13
5.3. ENCERRAMENTO E ANÚNCIO DO VENCEDOR.....	14
6. CONSIDERAÇÕES FINAIS.....	15

1. INTRODUÇÃO

Este trabalho descreve o desenvolvimento de uma implementação de uma versão reduzida do clássico jogo de Damas, utilizando o algoritmo de busca competitiva Minimax. O objetivo principal é aplicar conceitos de Inteligência Artificial para criar um agente capaz de tomar decisões estratégicas em um ambiente de soma zero. Nas seções seguintes, está detalhada formulação do problema, a lógica do pseudocódigo implementado e a interface visual do projeto através de capturas de tela.

2. O JOGO: TRADIÇÃO VS. ADAPTAÇÃO

Antes de começar a construir o jogo fisicamente, é fundamental entender seu funcionamento, conhecer suas regras e identificar as diferenças em relação à versão clássica. Para isso, é preciso analisar cuidadosamente as regras do jogo de damas convencional e compará-las com as alterações introduzidas nesta versão personalizada.

2.1. O JOGO DE DAMAS CLÁSSICO

O jogo de damas clássico é disputado em um tabuleiro 8x8, formado por 64 casas. Cada jogador começa com 12 peças, e o objetivo é capturar todas as peças do adversário ou imobilizá-las, de modo que não seja possível realizar nenhum movimento. Trata-se de um jogo de informação completa e soma zero.

2.2. VARIANTE "DAMAS DOS ELEMENTOS" (6X6)

A versão desenvolvida para este projeto introduz modificações estratégicas que alteram a árvore de decisão do algoritmo Minimax:

- **Dimensões do Tabuleiro:** O campo de jogo foi reduzido para uma matriz 6x6. Essa redução diminui o fator de ramificação, assim, permite que o algoritmo Minimax explore profundidades maiores em menos tempo.
- **Peças e Temática:** Inspirado nos elementos de *Avatar: The Last Airbender*, cada jogador possui 6 peças.

- **Composição Fixa:** 4 peças são atribuídas de forma fixa para o usuário e para o agente (ex: Água, Terra, Fogo e Ar).
- **Composição Aleatória:** 2 peças adicionais são sorteadas no início da partida, garantindo que cada jogo tenha um cenário inicial distinto.
- **Sistema de Pontuação (Pesos):** Diferente do jogo clássico, onde todas as peças têm o mesmo valor inicial, aqui cada elemento possui um peso específico. Isso exige que a função de avaliação do Minimax não conte apenas a quantidade de peças, mas a soma total do valor das peças remanescentes.
- **Mecânica de Movimento:** O jogador pode realizar o movimento de apenas uma peça por vez, seguindo a lógica de captura alternada.
- **Condição de Vitória:** O vencedor não é necessariamente quem elimina todas as peças, mas sim quem termina a partida com a maior pontuação acumulada, baseada nos pesos dos elementos que permaneceram ou foram capturados.

3. DINÂMICA DE PONTUAÇÃO E ANÁLISE DE CENÁRIOS

A inovação desta versão está na assimetria inicial das partidas. Ao contrário do jogo clássico, a presença de elementos aleatórios e pesos diferenciados aumenta significativamente a complexidade das decisões do algoritmo Minimax.

3.1. DISTRIBUIÇÃO DE PESOS E ALEATORIEDADE

Cada partida começa com uma configuração de forças diferente. As quatro peças fixas fornecem uma base estável, enquanto as duas peças aleatórias definem o foco da estratégia, seja ofensiva ou defensiva. A Tabela 1 apresenta um exemplo de possíveis pesos para os elementos, e a Tabela 2 ilustra um cenário real de jogo.

Tabela 1 - Exemplo de Atribuição de Pesos

Elemento	Peso (Pontos)	Quantidade
Dobra de Água	10	Fixa
Dobra de Terra	15	Fixa

Dobra de Fogo	20	Fixa
Dobra de Ar	25	Fixa
Elemento Extra A	Variável	Aleatória
Elemento Extra B	Variável	Aleatória

Tabela 2 - Exemplo de um possível jogo

Jogador	Peças Fixas (Peso Total)	Peças Aleatórias Sorteadas	Peso Total Inicial	Status de Vantagem
Usuário (Humano)	4 peças (70 pts)	2 x Ar (25+25)	120 pontos	Mais pontos (Desvantagem)
Agente (IA)	4 peças (70 pts)	2 x Água (10+10)	90 pontos	Menos pontos (Vantagem)

No exemplo mostrado na Tabela 2, a IA começa com menos pontos totais, enquanto o Humano possui peças mais valiosas. Isso coloca o Usuário (Humano) em desvantagem, pois há mais peças valiosas disponíveis para o agente de IA capturar e mais risco de perder suas próprias peças valiosas.

3.2. COMENTÁRIOS SOBRE A DINÂMICA DE JOGO

- **Desvantagem Estratégica:** Quando um jogador começa com menos pontos totais, o Minimax precisa avaliar cuidadosamente cada movimento. O algoritmo deve priorizar capturas que aumentem efetivamente o diferencial de pontos, em vez de simplesmente capturar o maior número de peças.
- **Assimetria de Risco:** Jogadores com peças mais valiosas têm maior pontuação, mas cada perda é mais significativa. Por exemplo, perder uma

peça de “Ar” (25 pts) impacta muito mais do que perder uma peça de “Água” (10 pts), mesmo que a quantidade de peças seja a mesma.

4. DETALHAMENTO DA IMPLEMENTAÇÃO

Nesta seção, o código implementado é analisado em duas frentes: a estratégia de busca (Minimax) e o critério de decisão (Função de Avaliação), demonstrando a aplicação prática da teoria estudada.

4.1. ALGORITMO DE BUSCA COMPETITIVA (MINIMAX)

A figura 1 apresenta a função principal do algoritmo. Esse trecho de código é responsável por navegar na árvore de possibilidades e alternar entre os turnos de maximização (IA) e minimização (Humano).

Figura 1 - Implementação do método minimax em Python.

```
1 def minimax(game: Game, depth: int, maximizing: bool, team: Team) -> tuple[float, Optional[Move]]:
2     """Minimax algorithm to determine the best move for a given game state."""
3     gamer_cloner = GameCloner()
4
5     if depth == 0 or game.is_game_over():
6         return evaluate(game, team), None
7
8     current_team = team if maximizing else team.opponent()
9     moves = game.get_all_moves(current_team)
10
11     if not moves:
12         return evaluate(game, team), None
13
14     best_move = None
15
16     if maximizing:
17         max_eval = -math.inf
18
19         for move in moves:
20             new_game = gamer_cloner.clone(game)
21             new_game.apply_move(move)
22
23             eval_score, _ = minimax(new_game, depth-1, False, team)
24
25             if eval_score > max_eval:
26                 max_eval = eval_score
27                 best_move = move
28
29         return max_eval, best_move
30
31     else:
32         min_eval = math.inf
33
34         for move in moves:
35             new_game = gamer_cloner.clone(game)
36             new_game.apply_move(move)
37
38             eval_score, _ = minimax(new_game, depth-1, True, team)
39
40             if eval_score < min_eval:
41                 min_eval = eval_score
42                 best_move = move
43
44         return min_eval, best_move
```

Visando validar a consistência da solução proposta, a implementação foi fundamentada no algoritmo MINIMAX-SEARCH, conforme estabelecido por Russell e Norvig em sua obra de referência sobre Inteligência Artificial. Diferente da versão básica do algoritmo, que pressupõe a exploração da árvore de estados até os nós terminais, este projeto incorpora a evolução necessária para lidar com a complexidade do jogo de damas.

Para otimizar o desempenho, o algoritmo foi adaptado para invocar a função de avaliação heurística (EVAL) por meio de um critério de Corte (Cut-off). Esta lógica substitui a verificação de estado terminal simplificada pela seguinte diretriz teórica:

O próximo passo é modificar a busca de modo que ela chame a função heurística EVAL quando for apropriado realizar o corte da busca.

Substituímos as linhas que mencionam **IS-TERMINAL** pela seguinte lógica: **if game.IS-CUTOFF(state, depth) then return game.EVAL(state, player), null**.
(Adaptado de Russell & Norvig, Capítulo 6).

Figura 2 - Algoritmo Minimax

```

function MINIMAX-SEARCH(game, state) returns an action
  player ← game.TO-MOVE(state)
  value, move ← MAX-VALUE(game, state)
  return move

function MAX-VALUE(game, state) returns a (utility, move) pair
  if game.IS-TERMINAL(state) then return game.UTILITY(state, player), null
  v, move ←  $-\infty$ 
  for each a in game.ACTIONS(state) do
    v2, a2 ← MIN-VALUE(game, game.RESULT(state, a))
    if v2 > v then
      v, move ← v2, a
  return v, move

function MIN-VALUE(game, state) returns a (utility, move) pair
  if game.IS-TERMINAL(state) then return game.UTILITY(state, player), null
  v, move ←  $+\infty$ 
  for each a in game.ACTIONS(state) do
    v2, a2 ← MAX-VALUE(game, game.RESULT(state, a))
    if v2 < v then
      v, move ← v2, a
  return v, move

```

Abaixo, está contido os relacionamentos dos componentes teóricos clássicos com as classes e métodos desenvolvidos neste projeto:

Tabela 3 - Tabela de Mapeamento Lógico

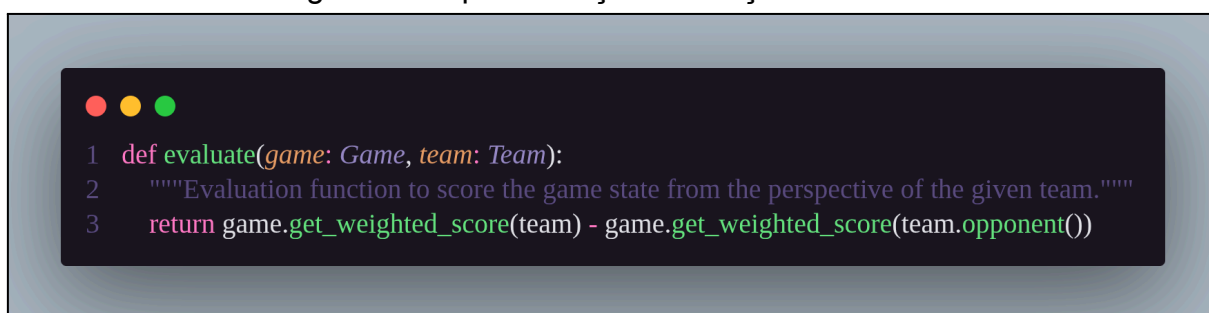
Elemento do Pseudocódigo (Russell & Norvig)	Implementação no Projeto (Python)	Descrição Técnica
MAX-VALUE(<i>game, state</i>)	minimax(..., maximizing=True)	Função que busca maximizar a utilidade (pontuação dos elementos) para a IA.
MIN-VALUE(<i>game, state</i>)	minimax(..., maximizing=False)	Função que simula a minimização da utilidade pelo oponente humano.

IS-TERMINAL(state)	game.is_game_over()	Verifica se o estado atual é o fim da partida (sem peças ou sem movimentos).
UTILITY(state, player)	evaluate(game, team)	Função de avaliação que calcula o saldo de pontos com base nos pesos.
ACTIONS(state)	game.get_all_moves(current_team)	Gera a lista de movimentos válidos (diagonais) para o estado atual.
RESULT(state, a)	new_game.apply_move(move)	Aplica uma ação a um estado clonado para gerar o próximo estado da árvore.

4.2. FUNÇÃO DE AVALIAÇÃO HEURÍSTICA (*HEURISTIC EVALUATION*)

A função de avaliação é o componente que permite à IA "entender" se ela está ganhando ou perdendo. No contexto deste projeto, ela substitui a contagem simples de peças pela soma ponderada dos elementos do Avatar.

Figura 3 - Implementação da função *evaluate*.



```

1 def evaluate(game: Game, team: Team):
2     """Evaluation function to score the game state from the perspective of the given team."""
3     return game.get_weighted_score(team) - game.get_weighted_score(team.opponent())

```

A lógica segue a equação matemática de soma zero:

Equação 1 - Função de Avaliação Heurística Ponderada

$$V = \sum P_{aliado} - \sum P_{adversário}$$

Onde P representa o peso de cada elemento vivo no tabuleiro. Essa abordagem garante que o Minimax não escolha apenas o caminho com mais capturas, mas sim

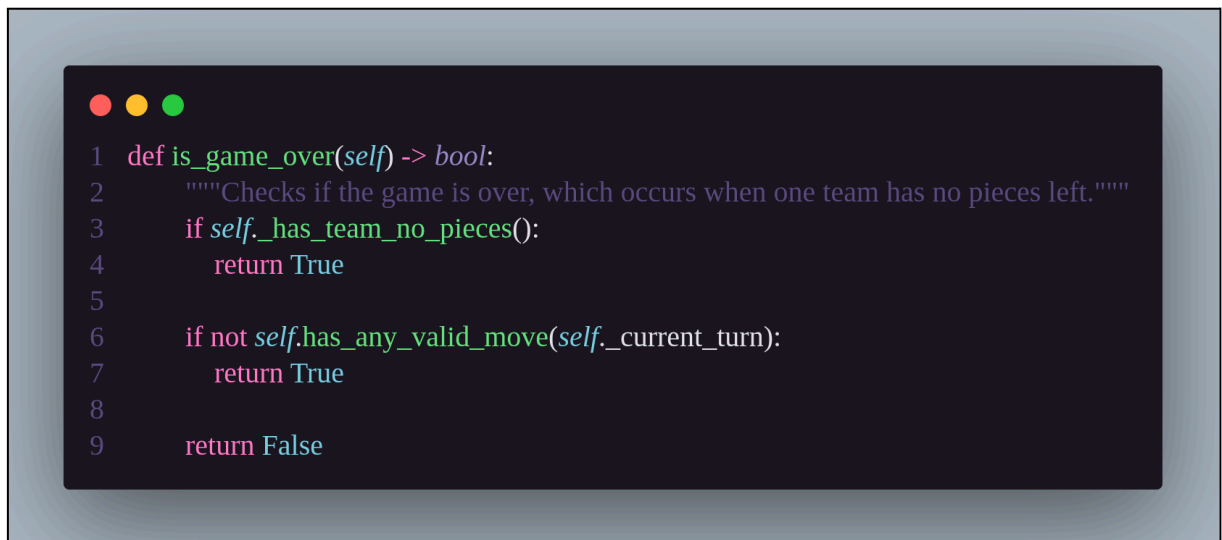
o caminho que protege as peças de maior valor (como o Ar) e elimina as ameaças mais caras do oponente.

4.3. CONDIÇÕES DE TERMINALIDADE (GAME OVER)

Para que o algoritmo Minimax possa interromper sua busca e retornar um valor de utilidade, é necessário identificar os estados terminais do jogo. Nesta implementação, a função `is_game_over` atua como gatilho de finalização.

A lógica mostrada na Figura 4 indica que, se não houver mais movimentos válidos para o jogador atual ou se uma equipe não tiver mais peças no tabuleiro, o jogo é encerrado. O vencedor é, então, o jogador que tiver a maior pontuação.

Figura 4 - Método de verificação de estado terminal.

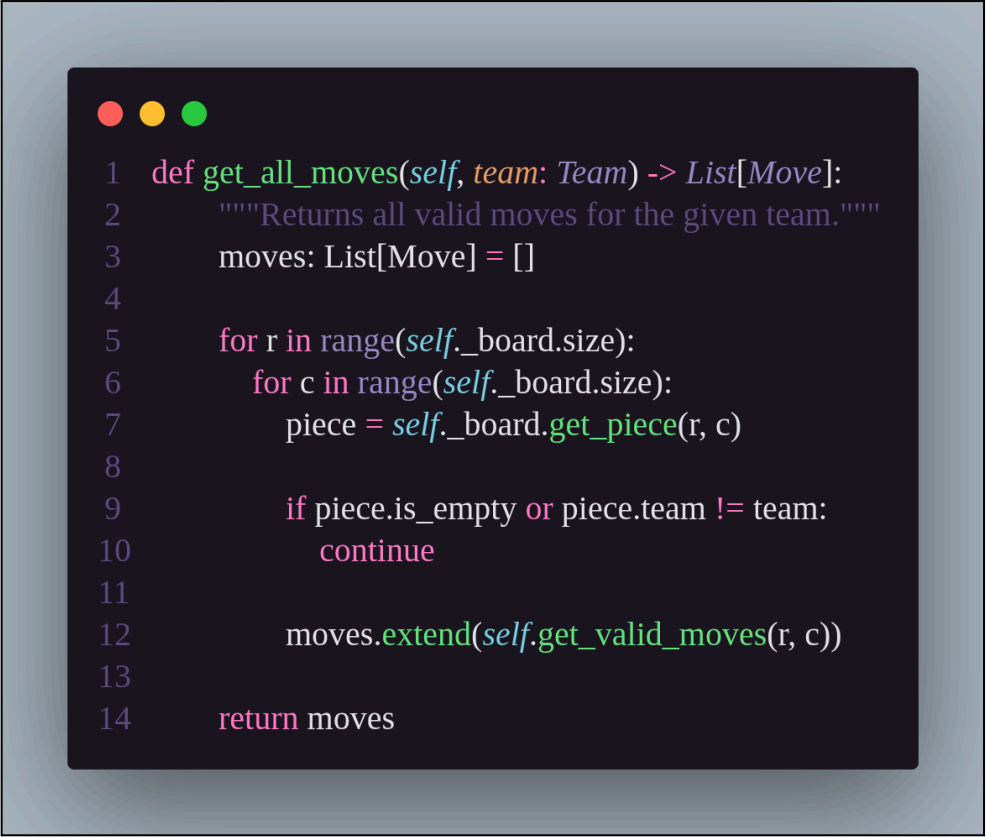


```
1 def is_game_over(self) -> bool:
2     """Checks if the game is over, which occurs when one team has no pieces left."""
3     if self._has_team_no_pieces():
4         return True
5
6     if not self.has_any_valid_move(self._current_turn):
7         return True
8
9     return False
```

4.5. Mecanismo de Exploração: Geração de Movimentos

Para que o algoritmo Minimax consiga projetar os estados futuros na árvore de decisão, é necessário um mecanismo eficiente de varredura do tabuleiro. O método `get_all_moves` cumpre este papel, atuando como o equivalente à função `ACTIONS(state)` da literatura de Russell & Norvig. Na figura 5 consta a implementação da função

Figura 5 - Implementação da varredura de movimentos válidos



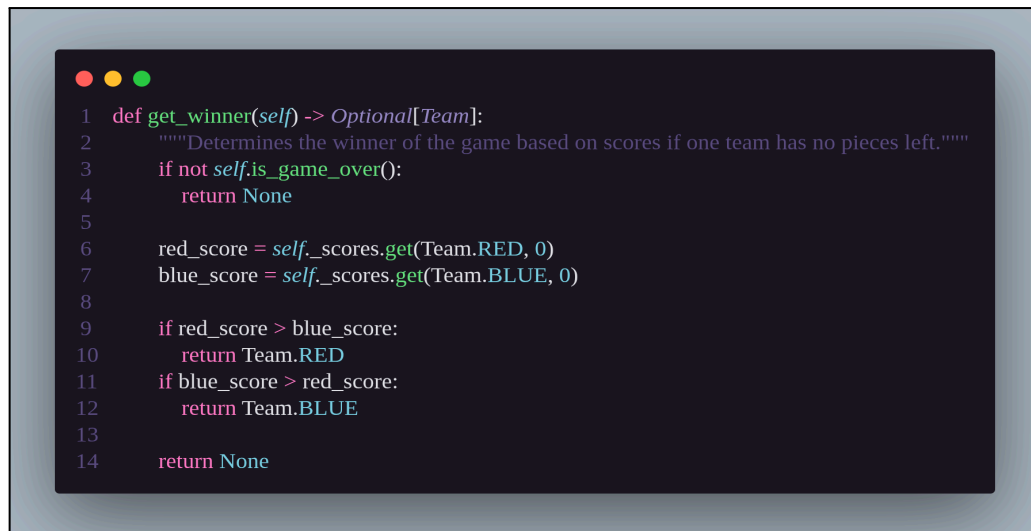
```
1 def get_all_moves(self, team: Team) -> List[Move]:
2     """Returns all valid moves for the given team."""
3     moves: List[Move] = []
4
5     for r in range(self._board.size):
6         for c in range(self._board.size):
7             piece = self._board.get_piece(r, c)
8
9             if piece.is_empty or piece.team != team:
10                continue
11
12            moves.extend(self.get_valid_moves(r, c))
13
14    return moves
```

4.4. DETERMINAÇÃO DO VENCEDOR POR PONTUAÇÃO PONDERADA

Ao contrário do jogo de damas tradicional, em que a vitória ocorre apenas quando todas as peças do adversário são capturadas, nesta versão o resultado depende do saldo de pontos acumulados. Cada elemento (Avatar) mantém um valor estratégico, tanto ao permanecer no tabuleiro quanto ao ser capturado, tornando a pontuação o critério principal para vencer.

A função `get_winner`, demonstrada na figura 6, é invocada assim que o estado terminal é atingido. Ela compara as pontuações finais de cada equipe para definir o resultado.

Figura 6 - Implementação da função de definição do vencedor.

A screenshot of a code editor with a dark background and light-colored text. The code is written in Python and defines a method `get_winner` for a class. The method takes `self` as an argument and returns an `Optional[Team]`. It includes a docstring, a check for `is_game_over`, score retrieval for RED and BLUE teams, and conditional logic to return the winning team or `None`.

```
1 def get_winner(self) -> Optional[Team]:
2     """Determines the winner of the game based on scores if one team has no pieces left."""
3     if not self.is_game_over():
4         return None
5
6     red_score = self._scores.get(Team.RED, 0)
7     blue_score = self._scores.get(Team.BLUE, 0)
8
9     if red_score > blue_score:
10        return Team.RED
11    if blue_score > red_score:
12        return Team.BLUE
13
14    return None
```

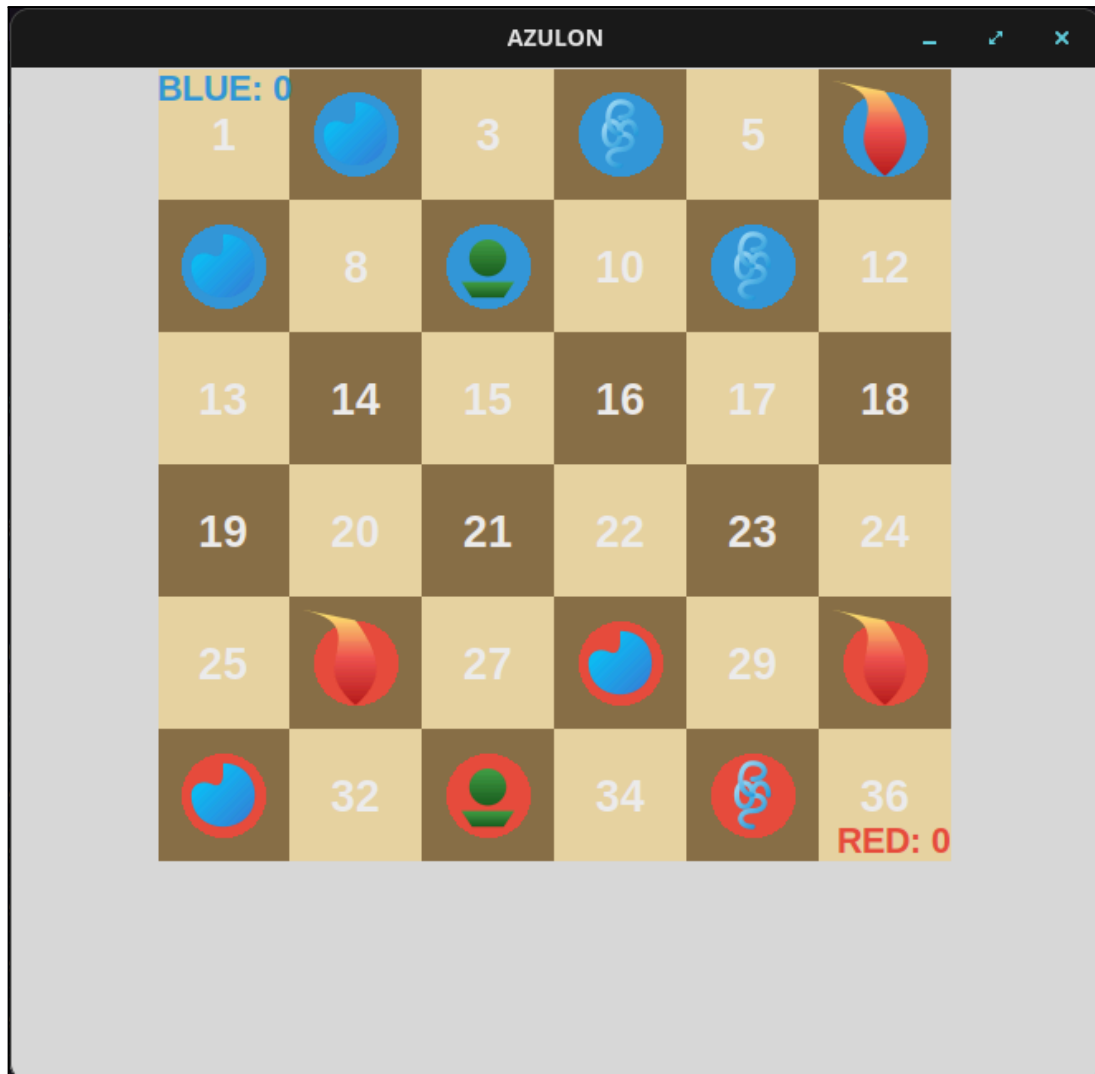
5. INTERFACE GRÁFICA E EXPERIÊNCIA DO USUÁRIO

A interface foi projetada para fornecer feedback em tempo real sobre o estado estratégico do jogo, permitindo que o usuário acompanhe não apenas as posições das peças, mas a evolução da pontuação ponderada.

5.1. AMBIENTE DE PARTIDA E MONITORAMENTO

O tabuleiro utiliza uma matriz 6x6 com casas de cores contrastantes, facilitando a visualização das diagonais. A Figura 7 mostra a tela inicial do jogo.

Figura 7 - Interface principal durante o jogo.

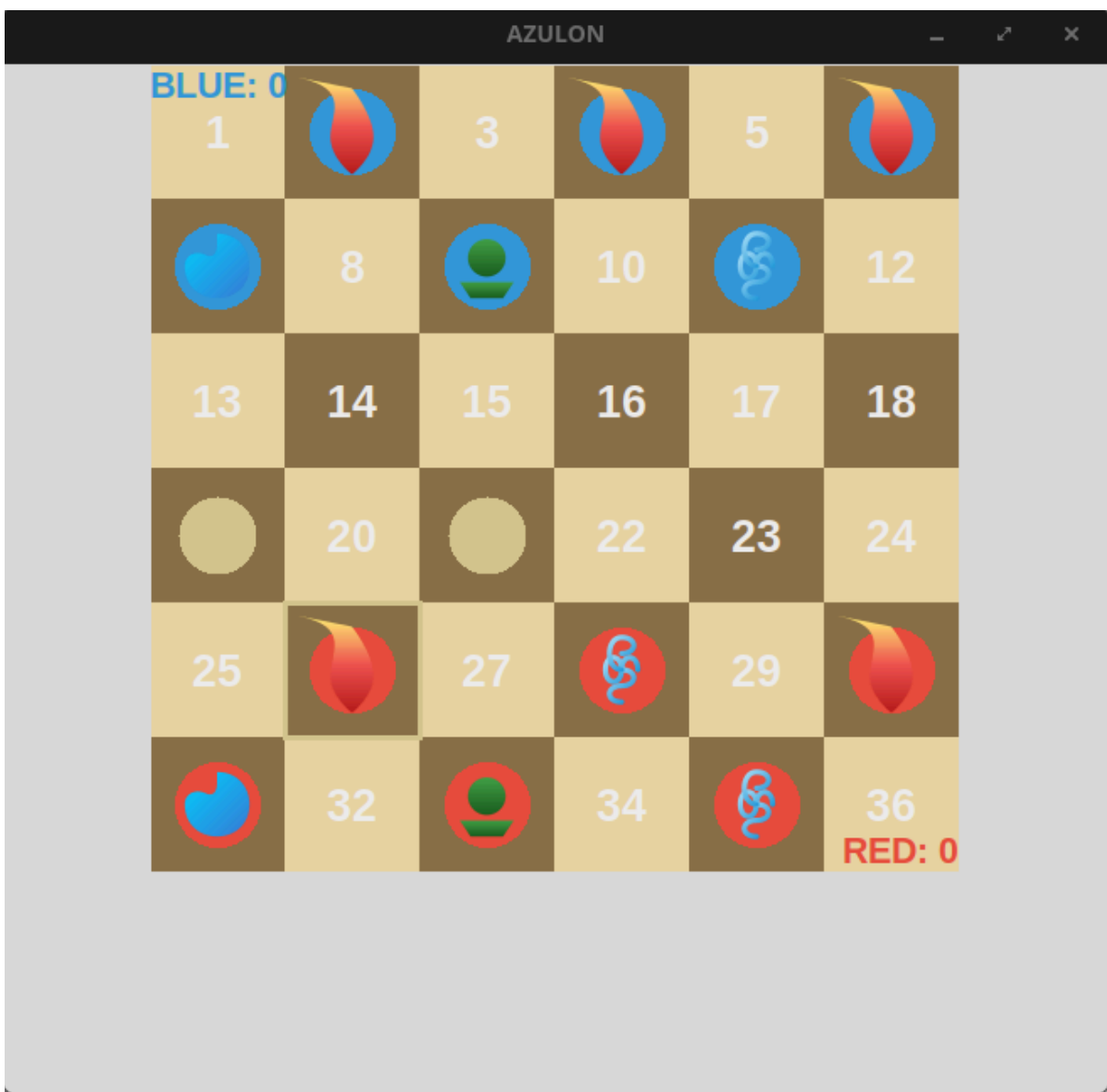


5.2. INDICAÇÃO DE MOVIMENTOS POSSÍVEIS

Para tornar a experiência de jogo mais intuitiva, a interface fornece um sistema de feedback visual imediato. Quando o usuário humano seleciona uma peça, o sistema calcula e destaca as opções disponíveis no tabuleiro.

A figura 8 demonstra como o sistema orienta o jogador durante o seu turno, mapeando as regras de movimentação diagonal para a matriz reduzida.

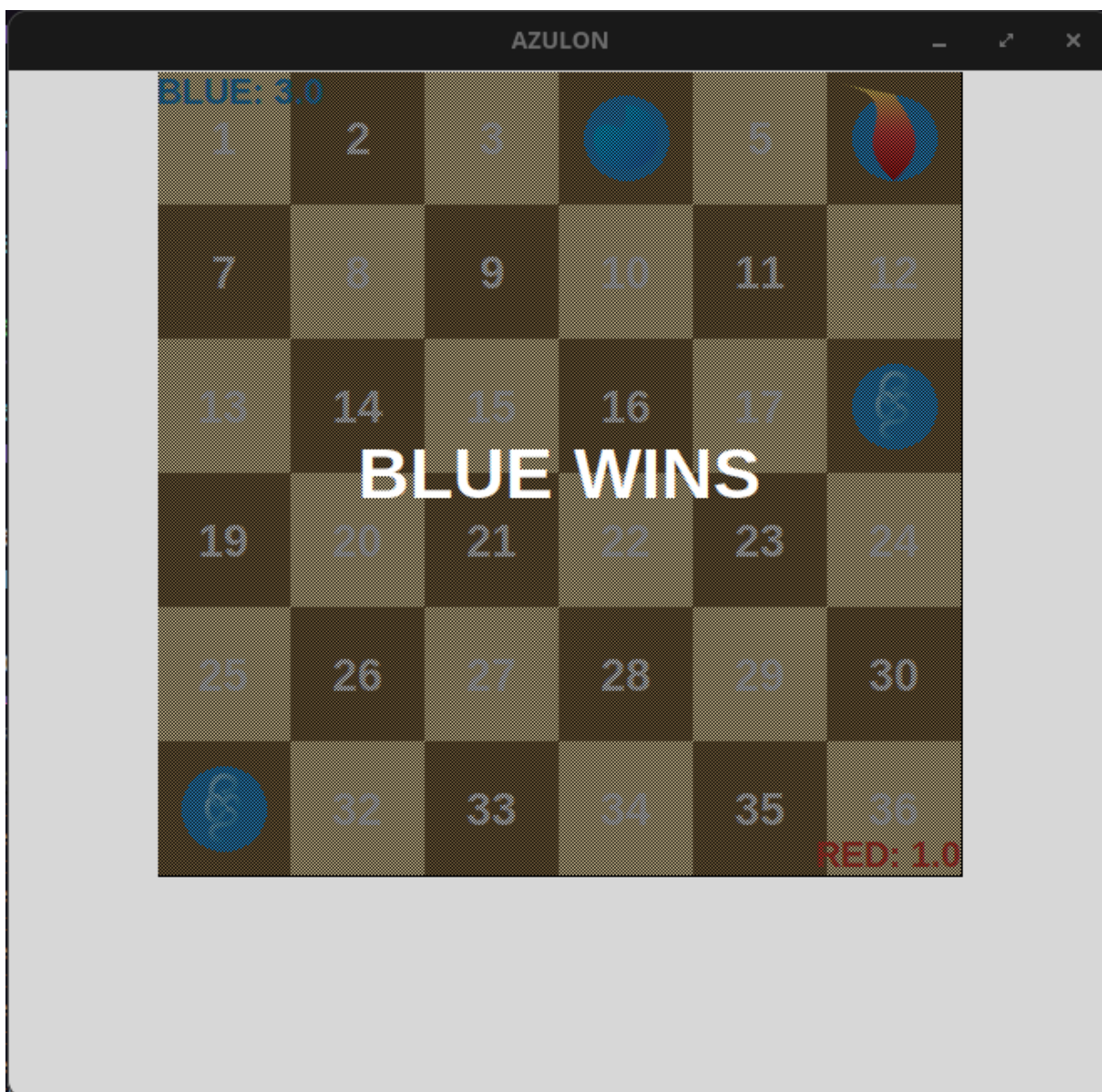
Figura 8 - Interface destacando as casas de destino válidas para o jogador humano.



5.3. ENCERRAMENTO E ANÚNCIO DO VENCEDOR

Ao atingir uma das condições de terminalidade descritas na Seção 4, o sistema interrompe a interação e sobrepõe uma tela de resultados. A Figura 9 mostra a tela de finalização do jogo.

Figura 9 - Tela de finalização (Game Over).



6. CONSIDERAÇÕES FINAIS

O desenvolvimento deste projeto possibilitou a aplicação prática de conceitos de Inteligência Artificial em um contexto de jogo competitivo. A adaptação do jogo de damas tradicional para uma versão inspirada no universo Avatar exigiu ajustes no algoritmo Minimax, que passou a considerar não apenas a quantidade de peças, mas também o valor estratégico de cada uma.

Durante o processo, a construção da interface gráfica mostrou-se a etapa mais trabalhosa, demandando diversas correções e refinamentos para garantir usabilidade e clareza visual. Apesar dos resultados obtidos, ainda há espaço para

melhorias, especialmente na interface e na otimização de alguns comportamentos do jogo.